



PROJET DE SYNTHÈSE

Plateforme de formation **MOS OFPPT**



Encadré par

Boujemaa el Rhazi



Réalisé par

Mohamed GHANIMI



Groupe

202



Module

projet de synthese



Année scolaire

2025 – 2026

1. Introduction et Contexte

L'Office de la Formation Professionnelle et de la Promotion du Travail (OFPPT) est le principal organisme marocain de formation professionnelle. Parmi ses certifications internationales, la certification Microsoft Office Specialist (MOS) valide la maîtrise de Word, Excel et PowerPoint (versions 2019 et 365) et est reconnue par les employeurs à l'échelle mondiale.

Aujourd'hui, la préparation à cette certification repose sur des supports physiques, des cours en présentiel et un suivi manuel — une approche qui limite la scalabilité, la traçabilité des apprentissages et la motivation des apprenants. Ce projet répond à ce besoin en développant une plateforme web fullstack moderne, gamifiée et adaptée au programme Certiport.

1.1 Problématique

- Absence d'un espace numérique centralisé pour la préparation MOS
- Suivi manuel de la progression des stagiaires, non scalable
- Manque de mécanismes de motivation (gamification, badges, streaks)
- Impossibilité de reproduire les conditions réelles de l'examen Certiport en ligne

1.2 Objectifs

- Centraliser le contenu pédagogique (Word, Excel, PowerPoint) dans un référentiel unique
- Évaluer les acquis via des quiz thématiques et des examens blancs chronométrés
- Motiver les apprenants par XP, badges conditionnels et classement (streaks)
- Offrir aux formateurs un espace de pilotage pédagogique et de génération de rapports
- Permettre aux administrateurs de gérer utilisateurs, contenu et statistiques globales

2. Architecture Technique

La plateforme adopte une architecture SPA (Single Page Application) découplée, basée sur la séparation stricte frontend/backend communiquant via une API REST JSON sécurisée.

Couche	Technologies	Rôle
Frontend	React 18 + TypeScript, Vite 5, Tailwind CSS v3, Zustand, React Router v6, Axios, shadcn/ui, Recharts	SPA — UX, navigation, gestion d'état, appels API
Backend	Laravel 11, PHP 8.3, Eloquent ORM, Sanctum, Form Requests, API Resources, Pest PHP	API RESTful sécurisée, logique métier, scoring, badges
Base de données	MySQL 8, Redis 7 (cache + queues)	Persistance, cache applicatif, files de traitement

Infrastructure	Docker Compose (MySQL, Redis, phpMyAdmin, MailHog)	Conteneurisation, environnement reproductible
CI/CD	GitHub Actions, Pest PHP, Vitest, Nginx (production)	Tests automatisés, déploiement continu

2.1 Sécurité

- Authentification Sanctum en double mode : Bearer token (mobile) et cookies httpOnly (SPA)
- Hashage bcrypt via Hash::make() — aucun mot de passe en clair en base
- Rate limiting : 5 tentatives/min sur /login et /register
- Middleware EnsureRole filtrant l'accès aux espaces Formateur et Admin
- Fichiers Office stockés dans storage/app/private/ — jamais accessibles publiquement
- CORS strict avec whitelist des origines autorisées

2.2 Performance

- Réponse API < 200 ms grâce à Redis comme cache applicatif
- Eager loading Eloquent (with, withCount) — élimination du problème N+1
- Score Lighthouse > 90 (Performance, Accessibilité, SEO) côté frontend
- Lazy loading React Router v6 — chargement à la demande des routes

3. Modèle de Données

Le modèle repose sur 14 entités Eloquent réparties en 5 packages logiques. Les conventions retenues : tables snake_case pluriel, clés étrangères suffixées _id, énumérations PHP 8.1 en VARCHAR majuscules, horodatages created_at / updated_at systématiques.

Package	Entités	Description
Identité	User	Profil, rôle (STUDENT/TRAINER/ADMIN), xp_points, streak_days, is_active
Pédagogie	Lesson, Exercise	Leçon liée à AppModule (WORD/EXCEL/POWERPOINT) et Difficulty (BEGINNER/INTERMEDIATE/ADVANCED)
Évaluation	Quiz, Question, QuizAttempt, ExamAttempt, ExamSession	Quiz (passing_score=70%, durée=15 min), tentatives tracées, mode examen strict avec UUID session
Suivi & Gamification	Progress, Enrollment, Badge, UserBadge	Progression par leçon, inscription module avec date cible, badges conditionnels + XP reward
Classes	ClassRoom, ClassMember	Classe virtuelle avec code invitation unique, gérée par un Formateur référent

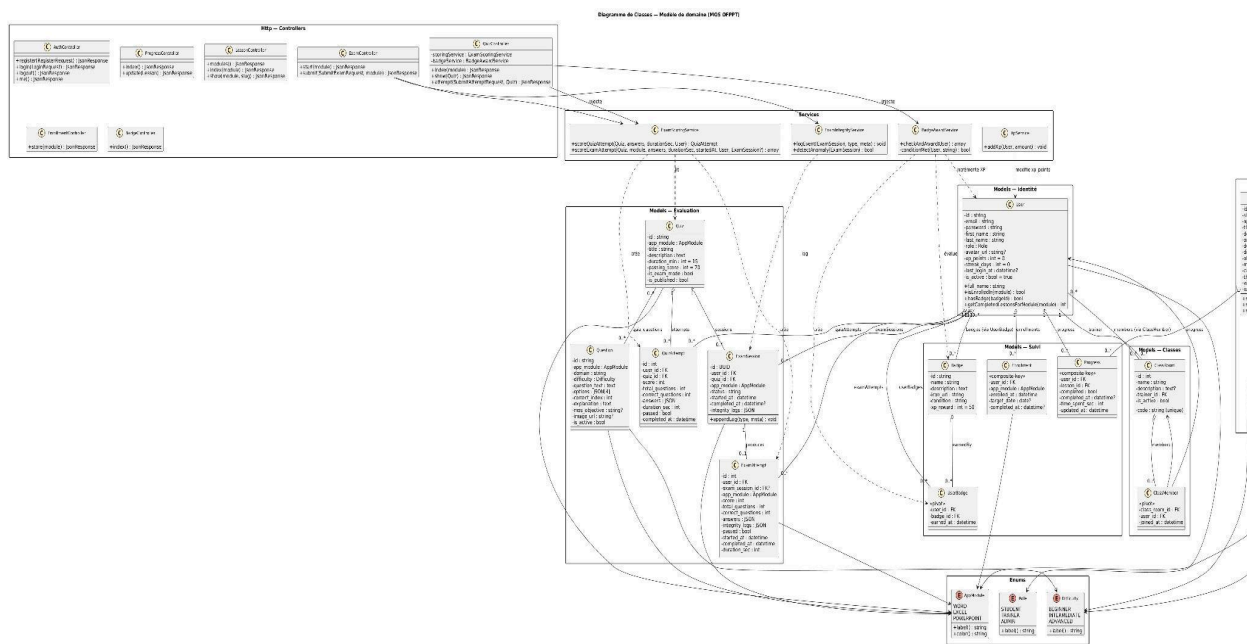


Figure 1 — Diagramme de classes UML (modèle de domaine MOS OFPPT)

4. Cas d'Utilisation

La plateforme est structurée autour de trois cas d'utilisation globaux couvrant les trois profils d'utilisateurs. Le visiteur s'inscrit et se connecte ; le stagiaire (héritier du visiteur) accède au parcours complet ; le formateur supervise ses classes ; l'administrateur gère l'ensemble de la plateforme.

Acteur	Périmètre principal	Cas d'utilisation clés
Visiteur	Accès public	S'inscrire, Se connecter, Réinitialiser mot de passe
Stagiaire	Apprentissage + Évaluation + Gamification	S'inscrire à un module, Lire leçons, Passer quiz, Examen blanc, Dashboard XP/badges
Formateur	Pilotage pédagogique	Gérer classes, Consulter étudiants, Exporter résultats CSV/PDF
Administrateur	Administration globale	Gérer utilisateurs, Publier contenu, Importer quiz JSON, Voir logs système
Système	Traitements automatiques	Calculer score, Attribuer XP, Débloquer badges, Réinitialiser streaks

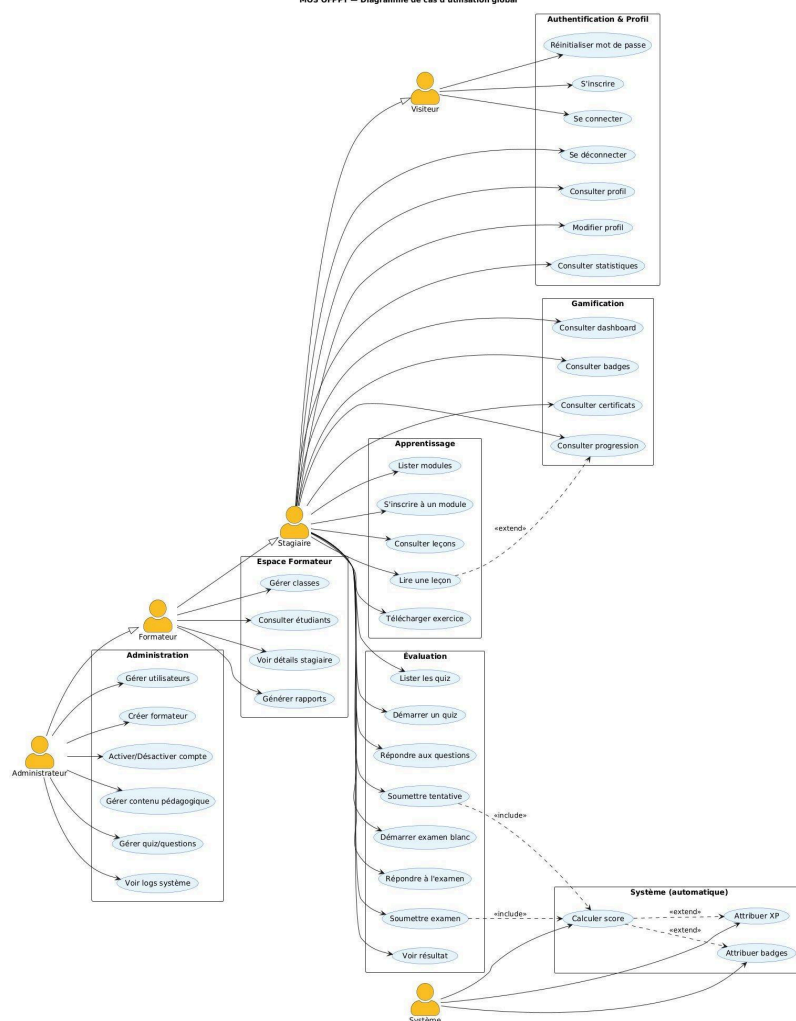


Figure 2 — Diagramme de cas d'utilisation global (MOS OFPPT)

5. Scénario Nominal — Soumission d'un Quiz

Le flux de soumission d'un quiz illustre l'orchestration complète des couches applicatives. Ce scénario est le plus critique du système : il déclenche le calcul du score, l'attribution des XP et la vérification des conditions de badges.

5.1 Étapes du flux

#	Acteur/Composant	Action
1	Stagiaire → Frontend	Clic « Soumettre le quiz » — récupération du token Sanctum dans authStore (Zustand)
2	Frontend → Middleware	POST /api/v1/quizzes/{quiz}/attempt avec Authorization: Bearer
3	Middleware auth:sanctum	Validation signature et expiration du token — transmet la requête

4	SubmitAttemptRequest	Validation payload : answers[] obligatoire, duration_sec entier ≥ 0 — erreur 422 si KO
5	QuizController	Injection de ExamScoringService et BadgeAwardService — appel scoreQuizAttempt()
6	ExamScoringService	Chargement questions (INNER JOIN quiz_questions), comparaison à correct_index, calcul score%
7	ExamScoringService	Persistance QuizAttempt (score, passed, answers, duration_sec, completed_at) en base MySQL
8	BadgeAwardService	Itération sur tous les badges — vérification conditionMet(user, badge.condition)
9	BadgeAwardService → XpService	Si condition remplie : INSERT user_badges + UPDATE users.xp_points += badge.xp_reward
10	QuizController → Frontend	Réponse JSON 200 OK : { data: QuizAttemptResource, newBadges: [], message }
11	Frontend	Mise à jour progressStore (Zustand) — affichage score + animation + modale badges

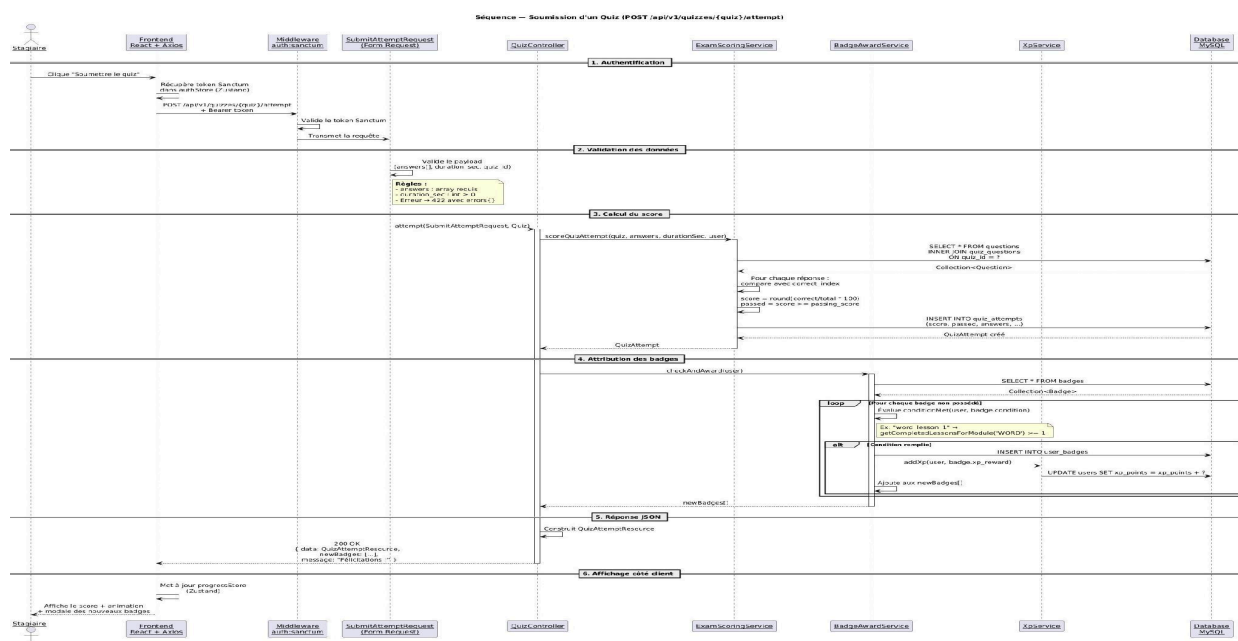


Figure 3 — Diagramme de séquence : Soumission d'un quiz (POST /api/v1/quizzes/{quiz}/attempt)

6. Modules Fonctionnels

6.1 Module Authentification & Profil

- Inscription avec validation de l'email (MailHog en dev)
- Connexion sécurisée — 5 tentatives/min (rate limiting Sanctum)
- Réinitialisation du mot de passe par lien email
- Gestion du profil : avatar, statistiques personnelles, changement de mot de passe

6.2 Module Apprentissage

- 3 modules pédagogiques : Word, Excel, PowerPoint — 3 niveaux de difficulté
- Inscription à un module avec date cible optionnelle (Enrollment)
- Leçons ordonnées avec marquage automatique de progression (Progress)
- Téléchargement d'exercices Office natifs (.docx, .xlsx, .pptx) — stockage privé Laravel

6.3 Module Évaluation

- Quiz thématiques chronométrés (15 min, 70% pour réussir) avec explications post-quiz
- Examen blanc en mode strict — sans correction affichée, UUID ExamSession pour intégrité
- Détection d'anomalies : changement d'onglet, perte de focus (integrity_logs JSON)
- Calcul automatique du score : $\text{round}(\text{correct}/\text{total} \times 100)$ comparé à passing_score

6.4 Module Gamification

- Points XP attribués par quiz réussi, leçon terminée, badge débloqué
- Badges conditionnels : ex. « Premier quiz Word réussi », « Score parfait examen »
- Streaks (jours de connexion consécutifs) — réinitialisés automatiquement après 24h d'inactivité
- Dashboard personnalisé : progression par module, badges obtenus, certificats

6.5 Espace Formateur & Administration

- Création de classes virtuelles avec code d'invitation unique (ex. ABC123)
- Suivi pédagogique : liste étudiants, progression, historique tentatives
- Export CSV/PDF des résultats — rapports globaux par classe
- Administration : création formateurs, activation/désactivation comptes, import quiz JSON, gestion badges

7. Exigences Non Fonctionnelles

Axe	Exigence
Sécurité	Bcrypt, Sanctum, rate limiting, CORS strict, middleware EnsureRole, stockage privé fichiers
Performance	API < 200 ms (Redis cache + eager loading), Lighthouse > 90, lazy loading React
Maintenabilité	SOLID, controllers fins, Form Requests, API Resources, énumérations PHP 8.1, tests Pest
Disponibilité	Docker Compose avec healthchecks MySQL/Redis, seeders reproductibles
Internationalisation	Interface française par défaut, dates d/m/Y, libellés localisés via label() des énumérations
Tests	Pest PHP (endpoints critiques), Vitest + React Testing Library (frontend), GitHub Actions CI

8. Roadmap & Livrables

8.1 Phases de développement

Phase	Nom	Contenu
Phase 1	MVP	Auth Sanctum, profil, leçons Word, quiz Word, dashboard minimal
Phase 2	Core	Modules Excel & PowerPoint, exercices, examen blanc, espace formateur + classes
Phase 3	Gamification	XP, badges conditionnels, streaks, analytics, notifications email
Phase 4	Optimisation	PWA, export PDF certificats, intégration API Certiport

8.2 Livrables attendus

- Code source backend Laravel (GitHub)
- Code source frontend React + TypeScript (GitHub)
- Fichiers JSON contenu pédagogique (Word, Excel, PowerPoint) — répertoire content/
- Migrations et seeders MySQL reproductibles
- Collection Postman / fichier OpenAPI documentant l'API REST
- Documentation Markdown : README, RULES.md, PRD.md
- Diagrammes UML PlantUML : cas d'utilisation, classes, séquence
- Environnement Docker Compose opérationnel en une commande

8.3 Critères d'acceptation

- Un stagiaire peut s'inscrire, se connecter, parcourir les leçons, télécharger un exercice, passer un quiz, obtenir un score et débloquent un badge
- Un formateur peut créer une classe, inviter des stagiaires et consulter leurs résultats
- Un administrateur peut publier une leçon, importer un quiz JSON et désactiver un compte
- Tous les tests Pest PHP des endpoints critiques passent au vert
- Le frontend ne présente aucune erreur TypeScript, respecte la checklist shadcn/ui
- Le temps de réponse API reste sous 200 ms en environnement local

9. Conclusion

La plateforme MOS OFPPT constitue une réponse complète et moderne aux besoins de digitalisation de la préparation à la certification Microsoft Office Specialist. En combinant une architecture SPA découplée (React + Laravel), un système de gamification motivant, et un espace de pilotage pédagogique pour les formateurs et administrateurs, ce projet apporte une valeur ajoutée significative à l'écosystème de formation de l'OFPPT.

L'adoption d'une stack technique actuelle (PHP 8.3, React 18, TypeScript strict, Docker, GitHub Actions) garantit la maintenabilité, la scalabilité et la sécurité de la solution. La roadmap en quatre phases permet une livraison incrémentale avec un MVP fonctionnel dès la première phase, évoluant vers une Progressive Web App intégrée à l'API Certiport en phase finale.

Ce projet, réalisé par l'équipe Trivolx dans le cadre du programme Bac+2 Développement Digital, démontre la capacité à concevoir, modéliser et développer une application fullstack de niveau professionnel, couvrant l'ensemble du cycle : analyse des besoins, modélisation UML, architecture logicielle, développement, tests et déploiement.